

CSC270 · DATABASE SYSTEMS · LECTURE 27

Hands-On Document Databases

MongoDB and the Document Model

CSC270 · Database Systems — COMSATS University Islamabad, Lahore Campus

Agenda

- 1 MongoDB architecture: mongod, databases, collections, documents
- 2 The document model vs. the relational row/table model
- 3 BSON — why binary JSON, not plain text
- 4 Core data types: ObjectId, dates, arrays, embedded documents
- 5 Modeling relationships by embedding vs. referencing
- 6 Real CRUD syntax: insert, find, update, delete
- 7 Query operators (\$gt, \$in, \$and) and projections
- 8 Realistic use cases for a document database

Recap: Where We Are

- Relational design fits well-structured data; MongoDB fits documents that vary in shape.
- Documents are BSON: richer types such as ObjectId and Date, plus faster traversal.
- Embed data you read together; reference data you query on its own.
- CRUD maps onto SQL: insert, find, update, delete — each with a One and a Many form.
- Next: how MongoDB scales, through replication and sharding.

Introduction to MongoDB

MongoDB is an open-source, document-oriented NoSQL database, first released in 2009. Instead of rows in tables, it stores data as documents — flexible, nested, JSON-like structures — grouped into collections.

2009

First released

Document

Data model — flexible, nested,
JSON-like

Rich query

Language + indexing, not just
get/set

ACID (v4.0+)

Multi-document transactions

BUILT FOR LECTURE 26'S PRIORITIES

MongoDB was built from the ground up around horizontal scaling and schema flexibility — while still offering a rich query language, indexing (the same core idea as Lecture 25, adapted to documents), and multi-document ACID transactions since version 4.0.

MongoDB Architecture

Clients — the mongosh shell, or a driver — connect to mongod (directly, or via a router in a cluster; Lecture 28) and issue commands against a four-level hierarchy:



mongod server process

One running MongoDB instance



Database

e.g. university



Collection

e.g. students, courses — analogous to a relational table



Document

e.g. one student's record — analogous to a relational row

Documents and Collections

mongosh

```
// A single document in the "students" collection
{
  _id: ObjectId("66f1a2b3c4d5e6f7a8b9c0d1"),
  regNo: "FA21-BCS-045",
  name: "Ayesha Khan",
  gpa: 3.72,
  department: "Computer Science",
  isActive: true,
  enrollmentDate: ISODate("2021-09-01")
}
```

RELATIONAL MODEL

Table

Fixed columns, every row has the same shape

DOCUMENT MODEL

Collection

No fixed columns, documents can vary in shape

TABLE → COLLECTION, ROW → DOCUMENT

A collection is MongoDB's rough analog of a table — but enforces no fixed schema: two documents can have entirely different fields. A document is the analog of a row — but its values can be arrays or nested documents, not just flat scalars.

BSON Data Format

MongoDB does not store documents as plain-text JSON. It stores them as BSON (“Binary JSON”) — designed for two things text JSON can't do well:

● Richer data types

Plain JSON only has strings, numbers, booleans, null, arrays, objects. BSON adds dates, binary data, and a dedicated 12-byte ObjectId type.

● Efficient traversal

Every BSON field is stored with a length prefix, so the database skips directly past fields it doesn't need instead of parsing character by character.

■ YOU NEVER WRITE BSON BY HAND

You write ordinary-looking JSON-like syntax in mongosh or a driver, and MongoDB encodes it to BSON internally, decoding it back to a readable representation whenever you view it.

MongoDB Data Types

Type	Example	Notes
ObjectId	ObjectId("66f1a2b3...")	12-byte unique identifier, auto-generated as _id
String	"Ayesha Khan"	UTF-8 text
Int32 / Int64	42, 9223372036854775807	Whole numbers, like a relational INT/BIGINT
Double	3.72	Default type for a decimal literal
Boolean	true, false	
Date	ISODate("2021-09-01")	Stored internally as ms since the Unix epoch
Array	["CSC270", "CSC211"]	An ordered list — values need not share a type
Embedded Document	{ street: "...", city: "..." }	A document nested as the value of a field

Embedding Related Data: One-to-Few

The biggest mental shift from the relational model: instead of splitting related data into separate tables, a document can embed related data directly inside the parent — trading normalization for reading everything about one entity in a single read.

A student with at most a handful of addresses — small, bounded, almost always read together with its parent:



```
db.students.insertOne({
  regNo: "FA21-BCS-045",
  name: "Ayesha Khan",
  addresses: [
    { type: "home", city: "Islamabad", street: "6 Lawrence St" },
    { type: "hostel", city: "Islamabad", street: "COMSATS Hostel Block C" }
  ]
});
```

mongosh

Embedding Related Data: One-to-Many

```
db.students.insertOne({
  regNo: "FA21-BCS-045",
  name: "Ayesha Khan",
  gpa: 3.72,
  enrolledCourses: [
    { courseCode: "CSC270" },
    { courseCode: "CSC211" }
  ]
});
```

mongosh

RELATIONAL (LECTURE 16)

Student table
regNo, name, gpa

Enrollment table
regNo (FK), courseCode (FK)

Course table
courseCode, title, creditHours

DOCUMENT APPROACH

One student document with enrolledCourses embedded directly as an array — reading a student's courses needs a two-table join relationally, but no join at all here.

Referencing Instead of Embedding

Embedding isn't always right. When related data is large, unbounded, or queried independently of its “parent” — many students sharing the same course — documents can instead reference each other by ObjectId, much closer to a relational foreign key:



mongosh

```
db.students.insertOne({
  regNo: "FA21-BCS-045",
  name: "Ayesha Khan",
  enrolledCourseIds: [ ObjectId("66f2b1c0..."), ObjectId("66f2b1c1...") ]
});
```

EMBED FOR TOGETHER, REFERENCE FOR INDEPENDENT

The relational model normalizes first and joins at query time; MongoDB asks you to design around your actual read pattern first — embed what you almost always read together, reference what you need to query, update, or share independently.

Inserting Documents



mongosh

```
db.students.insertOne({
  regNo: "FA21-BCS-046",
  name: "Bilal Ahmed",
  gpa: 3.15,
  department: "Computer Science",
  isActive: true
});

db.students.insertMany([
  { regNo: "FA21-BCS-047", name: "Sara Malik", gpa: 3.90,
    department: "Software Engineering", isActive: true },
  { regNo: "FA21-BCS-048", name: "Hamza Tariq", gpa: 2.85,
    department: "Computer Science", isActive: false }
]);
```

Querying Documents



mongosh

```
// Every document in the collection
db.students.find();

// Documents matching an exact value
db.students.find({ department: "Computer Science" });

// Just the first match
db.students.findOne({ regNo: "FA21-BCS-045" });
```

`find()` maps to SQL's `SELECT * ... WHERE`; `findOne()` returns a single document instead of a cursor.

Worked Example: Average GPA by Department



mongosh

```
// Stage 1 keeps active students; stage 2 groups and averages
db.students.aggregate([
  { $match: { isActive: true } },
  { $group: { _id: "$department", avgGpa: { $avg: "$gpa" } } },
  { $sort: { avgGpa: -1 } }
]);

// Each stage receives the output of the stage before it
```

Think of each stage as a SQL clause: `$match` is WHERE, `$group` is GROUP BY with AVG, and `$sort` is ORDER BY. Stages run left to right on the server.

Updating and Deleting Documents



mongosh

```
db.students.updateOne(  
  { regNo: "FA21-BCS-045" },  
  { $set: { gpa: 3.80 } }  
);  
  
db.students.deleteOne({ regNo: "FA21-BCS-048" });  
db.students.deleteMany({ isActive: false });
```

UPDATEONE/DELETEONE WITHOUT A UNIQUE FILTER IS A SILENT TRAP

They act on the first document MongoDB happens to match — if your filter isn't actually unique, you may not modify the document you meant to. Filter on a genuinely unique field (regNo, or _id) whenever you intend to touch exactly one document, the same discipline a relational WHERE on a primary key gives you.

Worked Example: Updating One Student Safely



mongosh

```
// Step 1: filter on a unique field, regNo
db.students.updateOne(
  { regNo: "FA21-BCS-045" },
  { $set: { gpa: 3.80 } }
);

// Result: matchedCount: 1, modifiedCount: 1 — verify before moving on
```

WHY THIS STEP ORDER

A unique filter guarantees exactly one document changes. Apply the same discipline to `deleteOne`: a shared field like `isActive` can silently match the wrong records.

Query Operators and Projections

mongosh

```
// $gt: greater than
db.students.find({ gpa: { $gt: 3.5 } });

// $in: value is one of a given set
db.students.find({ gpa: { $in: [3.5, 3.9] } });
```

Operator	Meaning	SQL equivalent
\$eq	Equal to	=
\$gt / \$gte	Greater than / or equal	> / >=
\$lt / \$lte	Less than / or equal	< / <=
\$in	Value is in a given list	IN (...)
\$and / \$or	Combine conditions	AND / OR

mongosh

```
// Projection: return only name and gpa (suppress _id)
db.students.find({ gpa: { $gt: 3.5 } }, { name: 1, gpa: 1, _id: 0 });
```

Worked Example: Projection Keeps Results Small



mongosh

```
// Without projection: the whole document comes back
db.students.find({ gpa: { $gt: 3.5 } });

// With projection: only the fields you need
db.students.find(
  { gpa: { $gt: 3.5 } },
  { name: 1, gpa: 1, _id: 0 }
);
```

The second argument mirrors SQL's SELECT column list. Smaller results mean less network traffic and faster responses.

MongoDB Use Cases

● Content management

Articles and pages that naturally vary in structure.

● Product catalogs

Wildly different attributes per category.

● User profiles

Flexible, evolving fields per user.

● Mobile / SPA backends

Document shape matches the app's own objects.

● Fast-evolving schemas

Active product development without migrations.

Key Takeaways

- MongoDB stores documents grouped into collections, served by mongod — roughly collection $\approx$ table, document $\approx$ row, but neither enforces a fixed shape.
- MongoDB stores data as BSON, not plain text JSON — for richer types (ObjectId, Date) and faster binary traversal.
- Model relationships by embedding (data almost always read together) or referencing an ObjectId (closer to a foreign key, better for independent queries).
- CRUD verbs — insertOne/insertMany, find/findOne, updateOne/updateMany, deleteOne/deleteMany — map directly onto SQL's INSERT, SELECT, UPDATE, DELETE.
- Query operators (\$gt, \$in, \$and) and projections give MongoDB most of SQL's WHERE/SELECT expressive power, in document form.